

Analyst Guide

Dashboard • Screener • API • Tear Sheets • Troubleshooting

Purpose

This guide explains how an analyst can use the Nifty 100 Analytics platform, including the Streamlit dashboard, financial screener, peer and sector views, API endpoints, generated reports and common troubleshooting procedures.

Project coverage

The platform covers company profiles, financial ratios, valuation, peer comparison, sectors, trends, capital allocation, documents and analytical reports for the Nifty 100 universe.

Guide version: Day 44 documentation

1. Getting Started

The project is intended to be run from the project root directory. The examples below assume a Windows PowerShell environment and the virtual environment used during testing.

Activate the virtual environment:

```
.\env\Scripts\Activate.ps1
```

Start the FastAPI service on port 8000:

```
uvicorn src.api.main:app --port 8000
```

Start the Streamlit dashboard on port 8501 from another terminal:

```
streamlit run src\dashboard\app.py --server.port 8501
```

Open the dashboard at **http://localhost:8501**. The API is available at **http://127.0.0.1:8000**. Both services can run at the same time because they use different ports.

Before analysis, confirm that the API health endpoint returns HTTP 200 and status=ok. This is also useful when diagnosing dashboard data-loading problems.

2. Dashboard Navigation

The Streamlit application contains eight screens. Use the sidebar/navigation area to move between them.

Home

The Home screen is the entry point. It introduces the platform and provides the navigation structure for the other screens.

Profile

Use Profile to inspect an individual company. Select a company/ticker and review its company information, financial metrics, valuation information and related analytical data.

Screener

Use Screener to filter companies using financial quality, growth, valuation and leverage criteria. Preset strategies can also be used when configured by the project.

Peers

Use Peers to inspect a company's peer group and compare important metrics against peer averages/benchmarks.

Trends

Use Trends for time-based financial analysis and growth trends, helping identify improving, stable or deteriorating performance.

Sectors

Use Sectors to compare broad sectors using company counts and summary financial metrics.

Capital

Use Capital to review cash flow, free cash flow, CFO quality, CapEx intensity and capital-allocation patterns.

Reports

Use Reports to access generated analytical outputs and company/sector reporting artifacts.

3. Using the Streamlit Screener

The screener combines filters from the project's financial-ratio dataset with company, sector and market data. Typical filters include ROE, debt-to-equity, free cash flow, revenue CAGR, PAT CAGR, operating margin, interest coverage, EPS CAGR, asset turnover, market capitalization, P/E, P/B, dividend yield, dividend payout, sales and net profit.

Basic workflow

1. Open the Screener screen.
2. Choose a preset if a predefined investment style is required.
3. Adjust individual filters when custom screening is required.
4. Review the resulting company list and quality score.
5. Open a company profile or peer comparison for deeper analysis.

Example API equivalent

```
curl -X GET "http://127.0.0.1:8000/api/v1/screener?min_roe=15" -H "accept: application/json"
```

The API response contains a count and a companies array. The Day 42 tests verified that the minimum-ROE filter returns companies meeting the requested threshold.

Important interpretation note

A screener is a filtering and ranking tool, not a recommendation engine by itself. Analysts should review the underlying financial statements, valuation, peer position and qualitative information before drawing conclusions.

4. Company Profiles and Peer Analysis

Company profiles provide a focused view of a selected Nifty 100 company. The API also exposes company-level data that can be used independently of the dashboard.

Example company API call

```
curl -X GET "http://127.0.0.1:8000/api/v1/companies/TCS" -H "accept: application/json"
```

An invalid company should return HTTP 404, which is covered by the API test suite.

Peer-group workflow

1. Select a company.
2. Identify its peer group.
3. Compare ROE, ROCE, revenue CAGR, PAT CAGR and other available metrics.
4. Check whether the company's metric is above or below the peer benchmark.
5. Use the comparison to identify relative strengths and weaknesses.

Example peer-group call

```
curl -X GET "http://127.0.0.1:8000/api/v1/peers/IT%20Services" -H "accept: application/json"
```

Example company peer comparison

```
curl -X GET "http://127.0.0.1:8000/api/v1/companies/TCS/peers/compare" -H "accept: application/json"
```

5. Sector and Trend Analysis

The Sectors screen summarizes the broad-sector structure of the database. The project's current sector data contains the broad sectors represented in the Nifty 100 universe.

Sector API

```
curl -X GET "http://127.0.0.1:8000/api/v1/sectors" -H "accept: application/json"
```

The response contains a count and sector-level records such as company count, median ROE, median P/E and median debt-to-equity.

Sector company example

```
curl -X GET "http://127.0.0.1:8000/api/v1/sectors/Information%20Technology/companies" -H "accept: application/json"
```

Trends screen

Use Trends to inspect how financial performance changes across years. Pay particular attention to revenue growth, PAT growth, EPS growth, profitability ratios and cash-flow quality. CAGR flags such as TURNAROUND, DECLINE_TO_LOSS, ZERO_BASE and INSUFFICIENT indicate edge cases rather than normal growth values.

6. Capital Allocation and Cash-Flow Intelligence

The Capital screen is designed to interpret how a company generates and deploys cash. Key measures include free cash flow, CFO/PAT quality, CapEx intensity and capital-allocation patterns.

Free Cash Flow

FCF is calculated as operating cash flow plus investing cash flow. Because investing cash flow is often negative, a negative FCF can be a valid result.

CFO quality

The CFO quality ratio compares total operating cash flow with PAT. A ratio above 1 is classified as High Quality, 0.5–1 as Moderate and below 0.5 as Accrual Risk. A zero PAT base returns no ratio.

Capital allocation patterns

The platform classifies combinations of operating, investing and financing cash-flow signs into labels such as Shareholder Returns, Reinvestor, Liquidating Assets, Distress Signal, Growth Funded by Debt, Cash Accumulator, Pre-Revenue and Mixed.

Analyst usage

Use this screen to distinguish growth investment from shareholder distributions, asset sales, debt-funded growth and potential distress signals. Interpret the pattern together with the company's balance sheet and profitability.

7. Generating and Using PDF Tear Sheets

The project produces company-level PDF tear sheets under **reports/tearsheets/**. The project also contains sector-level reports under **reports/sector/**.

Typical tear-sheet workflow

1. Ensure the SQLite database is populated and current.
2. Ensure the analytics outputs required by the report are generated.
3. Run the project's tear-sheet/report generation script from the project root.
4. Check the generated files under **reports/tearsheets/**.
5. Open several PDFs and verify company name, metrics, peer information and report dates before submission.

Because the exact generator filename can vary with the project revision, use the report-generation command documented in the project's current README or inspect the report-generation scripts before running it.

Quality checks

Confirm that there is one tear sheet per expected company, PDFs open without corruption, pages are complete, and the values match the SQLite/analytics outputs. Do not manually edit generated numbers; fix the source data or generator if a value is wrong.

Recommended locations

```
reports/tearsheets/ # company tear sheets
reports/sector/    # sector reports
```

8. API Reference and Example Calls

The API is exposed under `/api/v1/`. The following examples were verified during the API testing work.

Health

```
curl -X GET "http://127.0.0.1:8000/api/v1/health" -H "accept: application/json"
```

Companies

```
curl -X GET "http://127.0.0.1:8000/api/v1/companies" -H "accept: application/json"
curl -X GET "http://127.0.0.1:8000/api/v1/companies/TCS" -H "accept: application/json"
```

Screener

```
curl -X GET "http://127.0.0.1:8000/api/v1/screener?min_roe=15" -H "accept: application/json"
```

Sectors

```
curl -X GET "http://127.0.0.1:8000/api/v1/sectors" -H "accept: application/json"
```

Documents

```
curl -X GET "http://127.0.0.1:8000/api/v1/companies/TCS/documents" -H "accept: application/json"
```

The interactive OpenAPI definition is exported at **docs/openapi.json**, and the generated Postman collection is at **docs/postman_collection.json**.

9. Testing and Validation

The project has unit, data-quality, API and integration tests. Day 42 produced an HTML test report with 148 passing tests.

Run the complete test suite

```
python -m pytest tests\ -v
```

Generate the HTML report

```
python -m pytest tests\ --html=reports\pytest_report.html
```

Performance tests

```
python -m pytest tests\api\test_screener_load.py -v -s  
python -m pytest tests\api\test_profile_performance.py -v -s
```

Day 43 measured 10 concurrent screener requests with a total elapsed time of 0.0443 seconds and a maximum individual response time of 0.0432 seconds. Five company profile calls had a maximum response time of 0.0265 seconds. Both were comfortably below their targets.

The performance notes are stored in **output/perf_notes.md**. SQLite indexes were added to frequently queried company/year/date columns.

10. Troubleshooting

API port 8000 already in use

If Uvicorn reports WinError 10048, another FastAPI process is already using port 8000. Do not start a second copy. Test the existing service with:

```
Invoke-WebRequest http://127.0.0.1:8000/api/v1/health -UseBasicParsing | Select-Object  
StatusCode
```

Streamlit port 8501 already in use

Stop the previous Streamlit process or choose another port. For the Day 43 setup, keep FastAPI on 8000 and Streamlit on 8501.

ModuleNotFoundError: No module named 'src'

Run tests from the project root and prefer **python -m pytest** so the active virtual environment's Python interpreter is used.

No tests collected / import mismatch

Ensure test packages contain `__init__.py` where needed and remove stale `__pycache__` files if pytest reports an import-file mismatch.

Dashboard shows no data

Confirm the SQLite database exists at `data/db/nifty100.db`, FastAPI is healthy, and the dashboard is using the expected database/API configuration. Check the browser terminal output for the first failing query or request.

API returns 422 for an invalid typed query parameter

FastAPI performs type validation before the router function executes. If a test requires HTTP 400 for malformed input, explicitly handle or normalize that validation behavior at the API boundary.

11. Analyst Checklist

Before using the platform:

- Activate the project virtual environment.
- Confirm data/db/nifty100.db exists.
- Start FastAPI on port 8000.
- Start Streamlit on port 8501.
- Check /api/v1/health returns HTTP 200.
- Open the Home screen.
- Validate the required company profile.
- Run the screener and inspect filters.
- Compare selected companies with their peer group.
- Review sector and trend context.
- Review cash-flow and capital-allocation signals.
- Open supporting documents/annual reports where required.
- Review generated tear sheets and reports.

Before final submission:

- Run the full pytest suite.
- Generate the HTML test report.
- Run Black and Ruff checks.
- Confirm README setup/run instructions are current.
- Confirm docs/analyst_guide.pdf is present and readable.
- Confirm output/perf_notes.md is present.
- Confirm the final_deliverables archive contains the agreed 23 deliverables.

End of Analyst Guide